

Трансляция презентации (во время очных лекций)

<https://clck.ru/3D3Efj>



При просмотре презентации в PDF для отображения анимаций на слайдах необходимо использовать Acrobat Reader, KDE Okular, PDF-XChange, Foxit Reader, браузер Firefox. Для браузеров на движке Chrome (Edge, Яндекс, Opera, ...) необходимо использовать [PDF.js](#) с опцией «Enable active content (JavaScript) in PDFs».



Промышленное программирование

Тема 1

Системы сборки проектов на языке C/C++

Гаврилов Андрей Геннадьевич

кандидат технических наук, доцент

Кафедра Информационных технологий и вычислительных систем
МГТУ «СТАНКИН»

План лекции

1 Системы сборки программ

2 Cmake

Система сборки

Система сборки —

это программный инструмент или набор инструментов, который автоматизирует процесс преобразования исходного кода (а также связанных ресурсов, зависимостей и других файлов) в исполняемые программы или библиотеки.

Сборка (build) программ для C++ — это процесс, обычно включает в себя следующие ключевые этапы, управляемые системой сборки:

- **Препроцессинг:** обработка директив `#include`, макросов (`#define`) и условной компиляции (`#if`, `ifdef`, и т.д.).
- **Компиляция:** перевод исходного кода (`.cpp`, `.cxx` файлов) в объектные файлы (`.o`, `.obj`), содержащие машинный код.
- **Компоновка** (линковка): объединение всех объектных файлов и статических/динамических библиотек в единый исполняемый файл (`.exe`) или библиотеку (`.dll`, `.lib`, `.a`, `.so`).

Система сборки

Система сборки —

это программный инструмент или набор инструментов, который автоматизирует процесс преобразования исходного кода (а также связанных ресурсов, зависимостей и других файлов) в исполняемые программы или библиотеки.

Сборка (build) программ для C++ — это процесс, обычно включает в себя следующие ключевые этапы, управляемые системой сборки:

- **Препроцессинг**: обработка директив `#include`, макросов (`#define`) и условной компиляции (`#if`, `ifdef`, и т.д.).
- **Компиляция**: перевод исходного кода (`.cpp`, `.cxx` файлов) в объектные файлы (`.o`, `.obj`), содержащие машинный код.
- **Компоновка** (линковка): объединение всех объектных файлов и статических/динамических библиотек в единый исполняемый файл (`.exe`) или библиотеку (`.dll`, `.lib`, `.a`, `.so`).

Система сборки

Система сборки —

это программный инструмент или набор инструментов, который автоматизирует процесс преобразования исходного кода (а также связанных ресурсов, зависимостей и других файлов) в исполняемые программы или библиотеки.

Сборка (build) программ для C++ — это процесс, обычно включает в себя следующие ключевые этапы, управляемые системой сборки:

- **Препроцессинг**: обработка директив `#include`, макросов (`#define`) и условной компиляции (`#if`, `ifdef`, и т.д.).
- **Компиляция**: перевод исходного кода (`.cpp`, `.cxx` файлов) в объектные файлы (`.o`, `.obj`), содержащие машинный код.
- **Компоновка** (линковка): объединение всех объектных файлов и статических/динамических библиотек в единый исполняемый файл (`.exe`) или библиотеку (`.dll`, `.lib`, `.a`, `.so`).

Система сборки

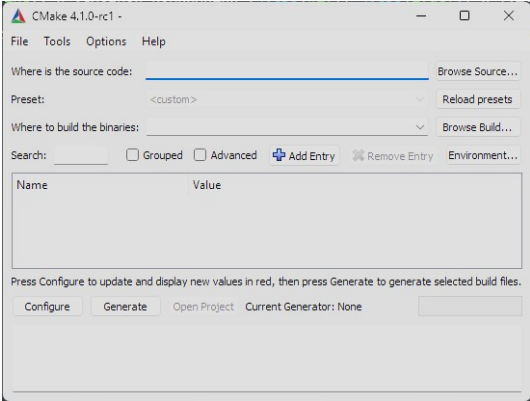
Система сборки —

это программный инструмент или набор инструментов, который автоматизирует процесс преобразования исходного кода (а также связанных ресурсов, зависимостей и других файлов) в исполняемые программы или библиотеки.

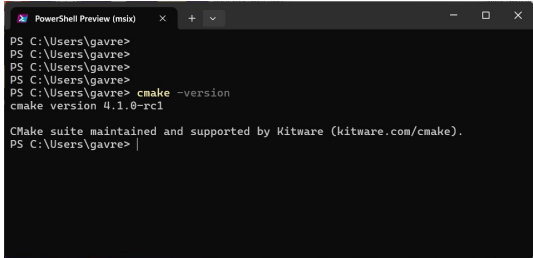
Сборка (build) программ для C++ — это процесс, обычно включает в себя следующие ключевые этапы, управляемые системой сборки:

- **Препроцессинг**: обработка директив `#include`, макросов (`#define`) и условной компиляции (`#if`, `ifdef`, и т.д.).
- **Компиляция**: перевод исходного кода (`.cpp`, `.cxx` файлов) в объектные файлы (`.o`, `.obj`), содержащие машинный код.
- **Компоновка** (линковка): объединение всех объектных файлов и статических/динамических библиотек в единый исполняемый файл (`.exe`) или библиотеку (`.dll`, `.lib`, `.a`, `.so`).

GUI



CLI



Соберём первую программу

1. Необходимые файлы

Файл с исходным кодом программы:

main.c

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Hello from CMake!");
6     return 0;
7 }
```

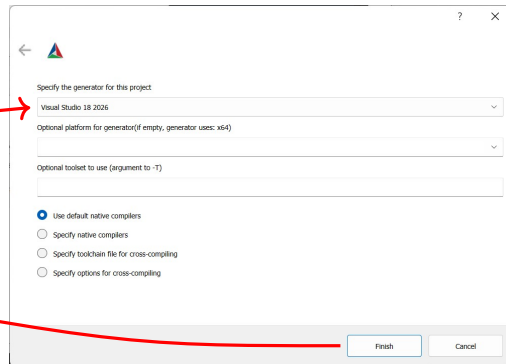
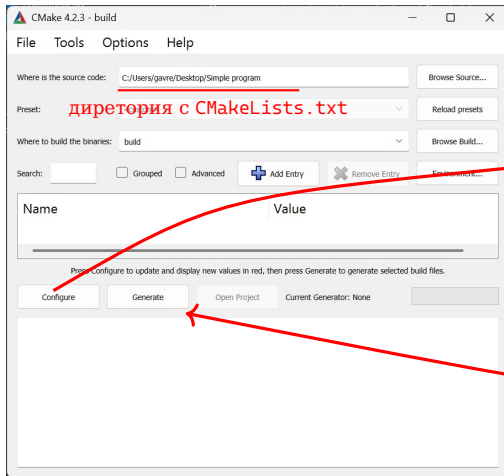
Файл с настройкой проекта:

CMakeLists.txt

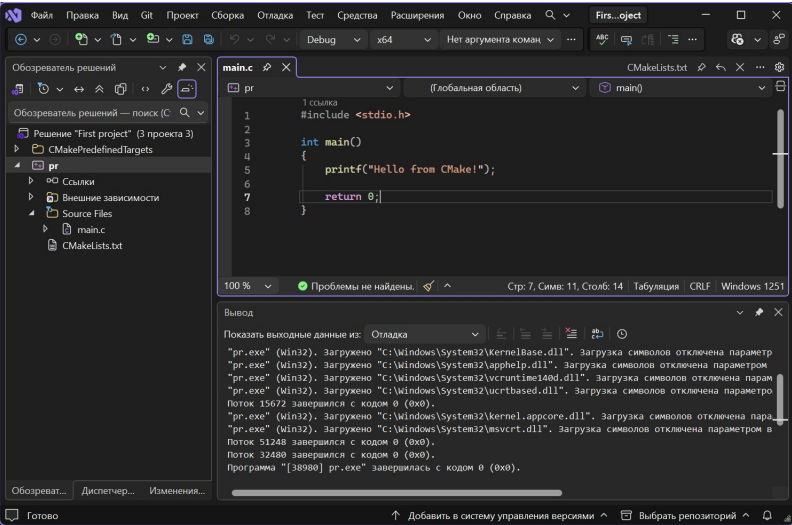
```
1 cmake_minimum_required(VERSION 4.1)
2 project(First_project)
3 add_executable(pr main.c)
```

Соберём первую программу

2. Работаем с Cmake



3. Результат



Структура папок

```
Sample program
├── CMakeLists.txt ←
├── main.c ←
└──
    └── L---build
        ├── ALL_BUILD.vcxproj
        ├── ALL_BUILD.vcxproj.filters
        ├── ALL_BUILD.vcxproj.user
        ├── CMakeCache.txt ←
        ├── ...
        └── ...
        └── +---Debug
            ├── pr.exe ←
            ├── pr.pdb
            └── ...
```

Проверим программу:

```
PowerShell 7.6.0-preview.6
```

```
PS Simple program> .\build\Debug\pr.exe
Hello from CMake!
```

```
PS Simple program>
```

Теперь то же самое в консоли (Windows)

```
PS C:\Users\gavre\Desktop\Simple program> rm -r build -Force
PS C:\Users\gavre\Desktop\Simple program> cmake -B build
-- Building for: Visual Studio 18 2026
-- Selecting Windows SDK version 10.0.26100.0 to target Windows 10.0.26200.
-- The C compiler identification is MSVC 19.50.35724.0
-- The CXX compiler identification is MSVC 19.50.35724.0
...
-- Build files have been written to: C:/Users/gavre/Desktop/Simple program/build

PS C:\Users\gavre\Desktop\Simple program> cmake --build build
...
pr.vcxproj -> C:\Users\gavre\Desktop\Simple program\build\Debug\pr.exe

PS C:\Users\gavre\Desktop\Simple program> .\build\Debug\pr.exe
Hello from CMake! ⇐
```

Теперь то же самое в консоли (Windows)

```
PS C:\Users\gavre\Desktop\Simple program> rm -r build -Force
PS C:\Users\gavre\Desktop\Simple program> cmake -B build
-- Building for: Visual Studio 18 2026
-- Selecting Windows SDK version 10.0.26100.0 to target Windows 10.0.26200.
-- The C compiler identification is MSVC 19.50.35724.0
-- The CXX compiler identification is MSVC 19.50.35724.0
...
-- Build files have been written to: C:/Users/gavre/Desktop/Simple program/build

PS C:\Users\gavre\Desktop\Simple program> cmake --build build
...
pr.vcxproj -> C:\Users\gavre\Desktop\Simple program\build\Debug\pr.exe

PS C:\Users\gavre\Desktop\Simple program> .\build\Debug\pr.exe
Hello from CMake!  ⇐
```

Теперь то же самое в консоли (Windows)

```
PS C:\Users\gavre\Desktop\Simple program> rm -r build -Force
PS C:\Users\gavre\Desktop\Simple program> cmake -B build
-- Building for: Visual Studio 18 2026
-- Selecting Windows SDK version 10.0.26100.0 to target Windows 10.0.26200.
-- The C compiler identification is MSVC 19.50.35724.0
-- The CXX compiler identification is MSVC 19.50.35724.0
...
-- Build files have been written to: C:/Users/gavre/Desktop/Simple program/build

PS C:\Users\gavre\Desktop\Simple program> cmake --build build
...
pr.vcxproj -> C:\Users\gavre\Desktop\Simple program\build\Debug\pr.exe

PS C:\Users\gavre\Desktop\Simple program> .\build\Debug\pr.exe
Hello from CMake!  ⇐⇐
```

И еще раз (Linux)

```
gavreg@GvrNoutiL: ... /Simple program$ cmake -B build_linux
-- The C compiler identification is GNU 14.2.0
-- The CXX compiler identification is GNU 14.2.0
...
-- Configuring done (6.4s)
-- Generating done (0.3s)
-- Build files have been written to: ... /Simple program/build_linux
gavreg@GvrNoutiL: ... /Simple program$ cmake --build build_linux
...
[100%] Built target pr
gavreg@GvrNoutiL: ... /Simple program$ tree build_linux/
build_linux/
├── CMakeCache.txt
├── Makefile
└── pr
...
gavreg@GvrNoutiL: ... /Simple program$ ./build_linux/pr
Hello from CMake!  ←←
```


И еще раз (Linux)

```
gavreg@GvrNoutiL: ... /Simple program$ cmake -B build_linux
-- The C compiler identification is GNU 14.2.0
-- The CXX compiler identification is GNU 14.2.0
...
-- Configuring done (6.4s)
-- Generating done (0.3s)
-- Build files have been written to: ... /Simple program/build_linux
gavreg@GvrNoutiL: ... /Simple program$ cmake --build build_linux
...
[100%] Built target pr
gavreg@GvrNoutiL: ... /Simple program$ tree build_linux/
build_linux/
├── CMakeCache.txt
├── Makefile
└── pr
...
gavreg@GvrNoutiL: ... /Simple program$ ./build_linux/pr
Hello from CMake!  ←←
```

И еще раз (Linux)

```
gavreg@GvrNoutiL: ... /Simple program$ cmake -B build_linux
-- The C compiler identification is GNU 14.2.0
-- The CXX compiler identification is GNU 14.2.0
...
-- Configuring done (6.4s)
-- Generating done (0.3s)
-- Build files have been written to: ... /Simple program/build_linux
gavreg@GvrNoutiL: ... /Simple program$ cmake --build build_linux
...
[100%] Built target pr
gavreg@GvrNoutiL: ... /Simple program$ tree build_linux/
build_linux/
├── CMakeCache.txt
├── Makefile
└── pr
...
gavreg@GvrNoutiL: ... /Simple program$ ./build_linux/pr
Hello from CMake!  ⇐⇐
```

Простые переменные

```
set(<variable> <value> ... [PARENT_SCOPE])
```

<variable> — имя переменной

<value> — значение переменной

PARENT_SCOPE — родительская область видимости

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.2)
2 project("First project")
3
4 set(A 1)
5 message("${A}")
6 set(A 1 2 3)
7 message("${A}")
8
9 add_executable(pr main.c)
```

```
PS ...> cmake -B build
...
1
1;2;3
...
```

Простые переменные

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.1)
2 project("First project")
3
4 set(X "ФГАОУ")
5 set(Y "ВО МГТУ")
6 set(Z "СТАНКИН")
7
8 message(${X} ${Y} ${Z})
9 message("${X} ${Y} ${Z}")
10
11 add_executable(pr main.c)
```

```
PS ... > cmake -B build
...
ФГАОУВО МГТУСТАНКИН
ФГАОУ ВО МГТУ СТАНКИН
...
-- Build files ...
```

Кешированные переменные

#с версии 4.2

```
set(CACHE{<variable>} [TYPE <type>] [HELP <helpstring> ... ]  
    [FORCE] VALUE [<value> ... ])
```

#до версии 4.2

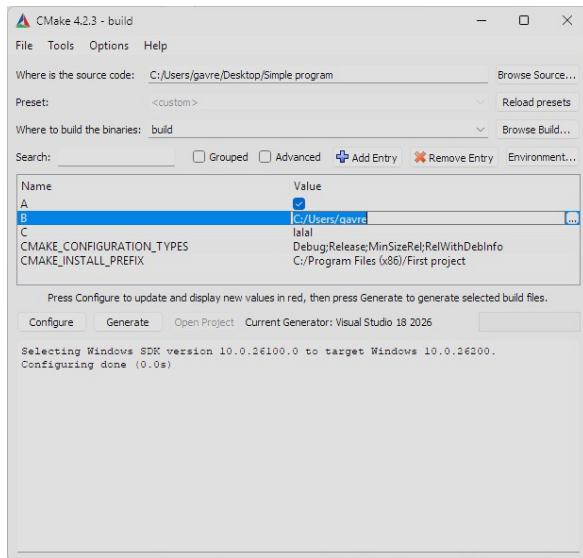
```
set(<variable> <value> ... CACHE <type> <docstring> [FORCE])
```

Тип	Описание
BOOL	Булево ON/OFF значение
FILEPATH	Путь к файлу диске
PATH	Путь к папке на диске
STRING	Строка
INTERNAL	Строка для внутреннего использовани

Установка кешированных переменных через GUI

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.2.3)
2 project("First project")
3
4 set (CACHE{A} TYPE BOOL
5     FORCE VALUE ON)
6 set (CACHE{B} TYPE FILEPATH
7     FORCE VALUE "C:/Users/gavre")
8 set (CACHE{C} TYPE STRING
9     FORCE VALUE "lalal")
10
11 add_executable(pr main.c)
```



Кешированные переменные. Пример

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.1)
2 project("First project")
3
4 set(X "ФГАΟΥ" CACHE STRING "Описание1")
5 set(Y "ΒΟ ΜΓΤΥ" CACHE STRING "Описание2")
6 set(Z "СТАНКИН" CACHE STRING "Описание3")
7 message("${X} ${Y} ${Z}")
8 message("${X} ${Y} ${Z}")
9
10 add_executable(pr main.c)
```

```
PS ... > cmake -B build
```

```
...
ФГАΟΥΒΟ ΜΓΤΥСТАНКИН
ФГАΟΥ ΒΟ ΜΓΤΥ СТАНКИН
...
-- Build files ...
```

```
PS ... > cat .\build\CMakeCache.txt
```

```
...
//Описание1
X:STRING=ФГАΟΥ

//Описание2
Y:STRING=ΒΟ ΜΓΤΥ

//Описание3
Z:STRING=СТАНКИН
...
```

Кешированные переменные. Пример

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.1)
2 project("First project")
3
4 set(X "ФГАΟΥ" CACHE STRING "Описание1")
5 set(Y "ΒΟ ΜΓΤΥ" CACHE STRING "Описание2")
6 set(Z "СТАНКИН" CACHE STRING "Описание3")
7 message(${X} ${Y} ${Z})
8 message("${X} ${Y} ${Z}")
9
10 add_executable(pr main.c)
```

```
PS ... > cmake -B build
```

```
...
```

```
ФГАΟΥΒΟ ΜΓΤΥСТАНКИН
```

```
ФГАΟΥ ΒΟ ΜΓΤΥ СТАНКИН
```

```
...
```

```
-- Build files ...
```

```
PS ... > cat .\build\CMakeCache.txt
```

```
...
```

```
//Описание1
```

```
X:STRING=ФГАΟΥ
```

```
//Описание2
```

```
Y:STRING=ΒΟ ΜΓΤΥ
```

```
//Описание3
```

```
Z:STRING=СТАНКИН
```

```
...
```


Изменение кешированных переменных

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.1)
2 project("First project")
3
4 message(${X} ${Y} ${Z})
5
6 add_executable(pr main.c)
```

```
PS ...> cmake -B build
...
ФГАОУ ВО МГТУ СТАНКИН
...
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.1)
2 project("First project")
3
4 set(Z "им. Баумана" CACHE STRING "Фууу")
5 message("${X} ${Y} ${Z}")
6
7 add_executable(pr main.c)
```

```
PS ...> cmake -B build
...
ФГАОУ ВО МГТУ СТАНКИН
...
PS ...> rm -r build -Force
PS ...> cmake -B build
...
им. Баумана
...
```

Изменение кешированных переменных

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.1)
2 project("First project")
3
4 message(${X} ${Y} ${Z})
5
6 add_executable(pr main.c)
```

```
PS ... > cmake -B build
...
ФГАОУ ВО МГТУ СТАНКИН
...
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.1)
2 project("First project")
3
4 set(Z "им. Баумана" CACHE STRING "Фууу")
5 message("${X} ${Y} ${Z}")
6
7 add_executable(pr main.c)
```

```
PS ... > cmake -B build
...
ФГАОУ ВО МГТУ СТАНКИН
...
PS ... > rm -r build -Force
PS ... > cmake -B build
...
им. Баумана
...
```

Изменение кешированных переменных

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.1)
2 project("First project")
3
4 message(${X} ${Y} ${Z})
5
6 add_executable(pr main.c)
```

```
PS ... > cmake -B build
...
ФГАОУ ВО МГТУ СТАНКИН
...
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.1)
2 project("First project")
3
4 set(Z "им. Баумана" CACHE STRING "Фууу")
5 message("${X} ${Y} ${Z}")
6
7 add_executable(pr main.c)
```

```
PS ... > cmake -B build
...
ФГАОУ ВО МГТУ СТАНКИН
...
PS ... > rm -r build -Force
PS ... > cmake -B build
...
им. Баумана
...
```

Изменение кешированных переменных. Force

CMakeLists.txt

```
1 ...
2 set (CACHE{A} VALUE "Адын!")
3 message("${A}")
4 set (CACHE{A} VALUE "Дуа!")
5 message("${A}")
6 set (CACHE{A} VALUE "Тури!")
7 message("${A}")
8 ...
```

```
PS ... > rm -r build -Force
PS ... > cmake -B build

...
Адын!
Адын!
Адын!
...
```

CMakeLists.txt

```
1 ...
2 set (CACHE{A} FORCE VALUE "Адын!")
3 message("${A}")
4 set (CACHE{A} FORCE VALUE "Дуа!")
5 message("${A}")
6 set (CACHE{A} FORCE VALUE "Тури!")
7 message("${A}")
8 ...
```

```
PS ... > cmake -B build

...
Адын!
Дуа!
Тури!
...
```

Изменение кешированных переменных. Force

CMakeLists.txt

```
1 ...
2 set (CACHE{A} VALUE "Адын!")
3 message("${A}")
4 set (CACHE{A} VALUE "Дуа!")
5 message("${A}")
6 set (CACHE{A} VALUE "Тури!")
7 message("${A}")
8 ...
```

```
PS ... > rm -r build -Force
PS ... > cmake -B build

...
Адын!
Адын!
Адын!
...
```

CMakeLists.txt

```
1 ...
2 set (CACHE{A} FORCE VALUE "Адын!")
3 message("${A}")
4 set (CACHE{A} FORCE VALUE "Дуа!")
5 message("${A}")
6 set (CACHE{A} FORCE VALUE "Тури!")
7 message("${A}")
8 ...
```

```
PS ... > cmake -B build

...
Адын!
Дуа!
Тури!
...
```

Переменные окружения

```
set(ENV{<variable>} [<value>])
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.2.3)
2 project("First project")
3
4 set (ENV{A} 1213)
5 message ("${ENV{A}}")
6
7 execute_process(COMMAND cmd /C "echo %A%")
8
9 message("${ENV{PATH}}")
10
11 add_executable(pr main.c)
```

```
PS ... > cmake -B build
```

```
...
```

```
Моя переменная = 1213
```

```
1213
```

```
C:\Program Files\WindowsApps\Microsoft  
PowerShell_7.5.4.0_x64__8wekyb3d8bbwe;C  
ProgramFiles\ImageMagick-7.1.1-Q16-HDR
```

```
...
```

Основные встроенные переменные CMake

Переменная	Описание	Пример
CMAKE_SOURCE_DIR	Корень исходников	/home/user/proj
CMAKE_BINARY_DIR	Корень сборки	/home/user/proj/build
CMAKE_CURRENT_SOURCE_DIR	Текущий исходник	/home/user/proj/src
CMAKE_VERSION	Версия CMake	3.27.1
CMAKE_SYSTEM	ОС	Linux, Windows
CMAKE_SYSTEM_NAME	Имя ОС	Linux
CMAKE_SYSTEM_PROCESSOR	Архитектура	x86_64
CMAKE_BUILD_TYPE	Тип сборки	Debug
CMAKE_INSTALL_PREFIX	Путь установки	/usr/local
PROJECT_NAME	Имя проекта	MyProject
PROJECT_SOURCE_DIR	Корень проекта	/home/user/proj
EXECUTABLE_OUTPUT_PATH	Путь для bin	/home/user/proj/build/bin
CMAKE_C_STANDARD	Стандарт C	90, 99, 11, 17, 23
CMAKE_CXX_STANDARD	Стандарт C++	98, 11, 14, 17, 20, 23
CMAKE_C_STANDARD_REQUIRED	Треб. стандарт C	TRUE / FALSE
CMAKE_CXX_STANDARD_REQUIRED	Треб. стандарт C++	TRUE / FALSE

Полный список: `cmake --help-variable-list`

Информация о переменной: `cmake --help-variable ИМЯ_ПЕРЕМЕННОЙ`

Встроенные переменные связанные с компилятором

Переменная	Описание	Пример
CMAKE_C_COMPILER	Компилятор C	/usr/bin/gcc
CMAKE_CXX_COMPILER	Компилятор C++	/usr/bin/g++
CMAKE_COMPILER_IS_GNU_CC	GNU C компилятор?	TRUE / FALSE
CMAKE_COMPILER_IS_GNU_CXX	GNU C++ компилятор?	TRUE / FALSE
CMAKE_C_COMPILER_ID	Идент-ор. C компилятора	GNU, Clang, MSVC
CMAKE_CXX_COMPILER_ID	Идент-ор. C++ компилятора	GNU, Clang, MSVC
CMAKE_C_COMPILER_VERSION	Версия C компилятора	11.4.0
CMAKE_CXX_COMPILER_VERSION	Версия C++ компилятора	11.4.0
CMAKE_C_FLAGS	Флаги для C компилятора	-O2 -Wall
CMAKE_CXX_FLAGS	Флаги для C++ компилятора	-O2 -Wall -std=c++11
CMAKE_C_FLAGS_DEBUG	Флаги для Debug (C)	-g
CMAKE_CXX_FLAGS_DEBUG	Флаги для Debug (C++)	-g
CMAKE_C_FLAGS_RELEASE	Флаги для Release (C)	-O3 -DNDEBUG
CMAKE_CXX_FLAGS_RELEASE	Флаги для Release (C++)	-O3 -DNDEBUG
CMAKE_EXE_LINKER_FLAGS	Флаги линковщика	-static -lpthread
CMAKE_LINKER	Линковщик	/usr/bin/ld

Условия

```
1 if(<условие>)
2   Команды
3 elseif(<другое условие>) # optionally. can repeat
4   Команды
5 else()                    # optionally
6   Команды
7 endif()
```

`if(<константа>)`

Истина — 1 ON YES Y TRUE $\neq$ 0

Ложь — 0 OFF NO FALSE N IGNORE NOTFOUND "..."NOTFOUND"

`if(<переменная>)`

Истина — существующая переменная, не являющаяся ложностью константой

Ложь — во всех остальных случаях

`if("<строка>")`

Истина — Если является истинной константой

Ложь — во всех остальных случаях

Логические операторы

`if(NOT <условие>)` — отрицание

`if(<условие1> AND <условие2>)` — логическое И

`if(<условие1> OR <условие2>)` — логическое ИЛИ

`if((условие1) AND (условие2 OR (услвие3)))` — сложные условия

Компараторы

```
if(<variable|string> XXX <variable|string>)
```

XXX:

LESS	—	строго меньше
GREATER	—	строго больше
EQUAL	—	равно
LESS_EQUAL	—	меньше или равно (3.7+)
GREATER_EQUAL	—	больше или равно (3.7+)
STRLESS	—	лексикографически меньше
STRGREATER	—	лексикографически больше
STREQUAL	—	лексикографически равно
STRLESS_EQUAL	—	лексикографически меньше или равно (3.7+)
STRGREATER_EQUAL	—	лексикографически больше или равно (3.7+)
MATCHES	—	соответствует регулярному выражению (группы () сохраняются в CMAKE_MATCH_<n> (2.6+))

Сравнение версий и путей

```
if(<variable|string> XXX <variable|string>)
```

Сравнение версий:

VERSION_LESS	—	строго меньше
VERSION_GREATER	—	строго больше
VERSION_EQUAL	—	равно
VERSION_LESS_EQUAL	—	меньше или равно (3.7+)
VERSION_GREATER_EQUAL	—	больше или равно (3.7+)

Формат версии: major[.minor[.patch[.tweak]]], пропущенные компоненты = 0

Сравнение путей (3.24+):

PATH_EQUAL — покомпонентное сравнение путей

Коллапсирование разделителей: `if("/a//b/c" PATH_EQUAL "/a/b/c") = true`

Циклы. Foreach

```
foreach(<loop_var> <items>)  
<commands>  
endforeach()
```

CMakeLists.txt

```
1 foreach(A "Раз два три")  
2 message("Итерация = ${A}")  
3 endforeach()  
4  
5 foreach(A Раз два три)  
6 message("Итерация = ${A}")  
7 endforeach()  
8  
9 set(X Вася Петя Маша)  
10  
11 foreach(A ${X})  
12 message("Имя = ${A}")  
13 endforeach()
```

```
PS ...> cmake -B build  
...  
Итерация = Раз два три  
Итерация = Раз  
Итерация = два  
Итерация = три  
Имя = Вася  
Имя = Петя  
Имя = Маша  
...
```